



CheckAR

AI-Powered Smart Car Diagnostics — dashboard-light and engine-sound diagnosis without OBD-II hardware

GROUP 11

Funwi Kelsea Ndohnwi FE23A066

Ngadang Assonfack P. Estelle FE23A109

Ateh Swirri Fofang FE23A018

Tabot Ghislain Orock FE23A146

Diagnosing a fault shouldn't require a mechanic visit

Modern vehicles pack dozens of interdependent electronic systems. When a dashboard light flashes or an engine makes an unfamiliar sound, most car owners — especially across Cameroon and similar markets — have no way to interpret it themselves.



Unnecessary mechanic visits

Even minor, self-diagnosable issues get escalated, costing time and money.



Risk of exploitation

Non-technical owners are vulnerable to unscrupulous repairers.



Hardware-dependent tools

Existing OBD-II scanners need a physical dongle many older cars lack.



Two problems. One unified fix.

Dashboard warning-light image

Photograph a lit icon; CheckAR recognises it instantly.

Engine-sound recording

Record up to 10 seconds; on-device ML classifies the fault.

No OBD-II dongle required

Works with just a smartphone camera and microphone.

General & specific objectives



General objective: Design and specify CheckAR, an AI-powered app enabling car owners and mechanics to diagnose common faults from images and sound — no OBD-II hardware needed.

Task 1

Review mobile dev. approaches & pick an architecture

Task 2

Identify stakeholders & gather real requirements

Task 3

Analyze, prioritize & specify requirements (SRS)

Task 4

Model system scope & behaviour with diagrams

Task 5

Design a minimal, easy-to-implement UI/UX

Task 6

Design the data model & backend architecture

Why existing OBD-II scanner apps fall short

Capability	FIXD OBD2 Scanner	Car Scanner ELM OBD2	CheckAR
Works without a physical dongle	X	X	✓
Diagnoses from a photo of a warning light	X	X	✓
Diagnoses from engine sound	X	X	✓
Plain-language explanation + urgency level	Partial	Partial	✓
Mechanic discovery & tutorial links	X	X	✓



Insight: hardware-free, image- and sound-based diagnosis with plain-language guidance is the gap CheckAR is built to fill.

From interviews to a prioritized SRS

HOW REQUIREMENTS WERE GATHERED

- Structured interviews with owners & mechanics
- Bilingual (EN/FR) Google Forms surveys
- Group brainstorming sessions
- Reverse engineering of FIXD & Car Scanner

User-reluctance patterns uncovered

Accuracy-trust paradox · technical intimidation ·
safety/liability worries · poor lighting & connectivity

SAMPLE FUNCTIONAL REQUIREMENTS (MoSCoW)

ID	Requirement	Priority
FR5	Capture dashboard images & detect warning-light icons	Must-Have
FR7	Record engine sound clips up to 10 seconds	Must-Have
FR8	Classify engine sounds into fault categories	Must-Have
FR13	Operate offline using on-device models	Must-Have
FR10	Link to relevant video tutorials per issue	Should-Have
FR17	Estimate urgency: Immediate / Soon / Monitor	Must-Have

Modelling scope and behaviour (Task 4)

Context Diagram

CheckAR exchanges data with the Admin, User, Google Maps, YouTube, and an Authentication Service.



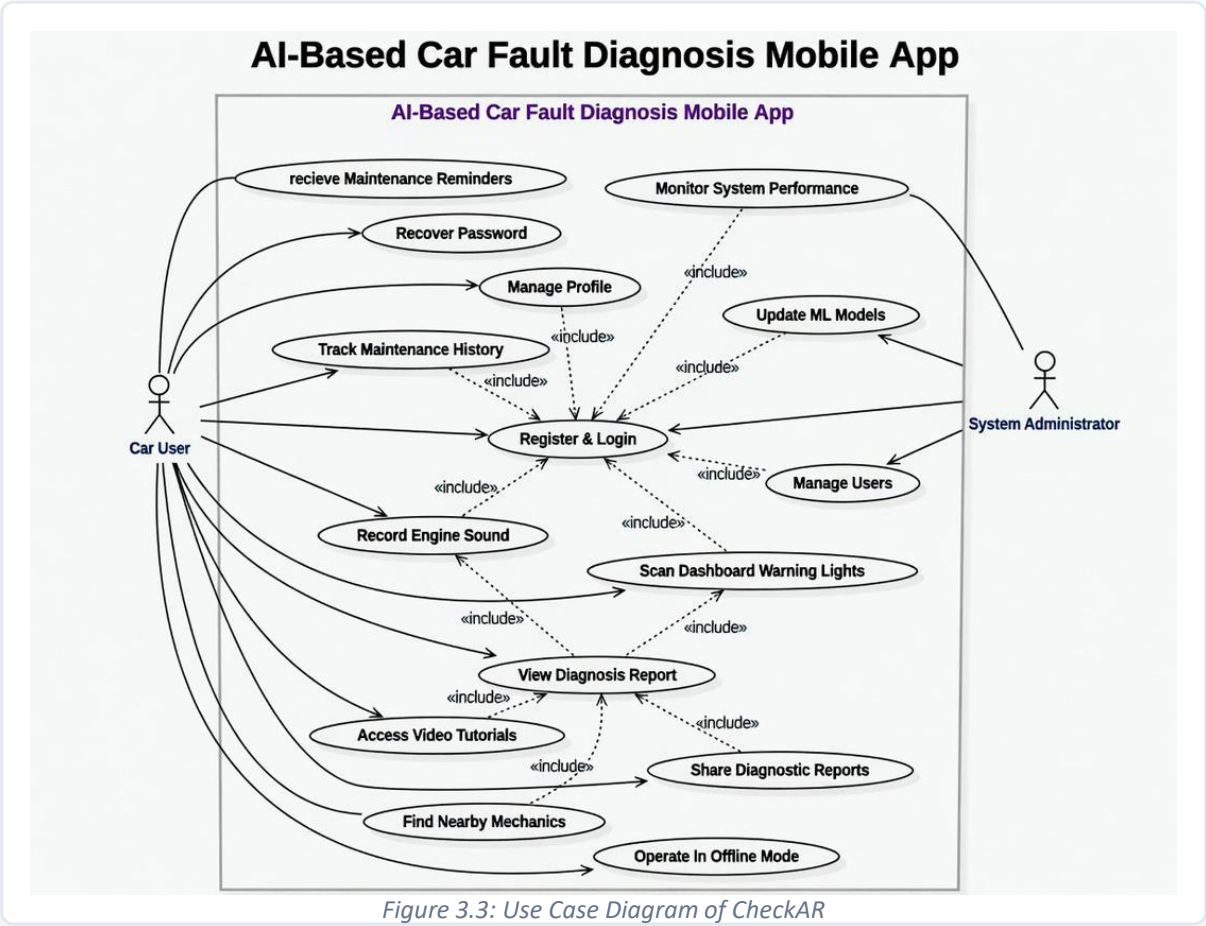
Data Flow Diagram

Decomposes the system into authentication, image diagnosis, audio diagnosis, reporting, and maintenance-tracking processes.



Use Case Diagram

Two actors — Car User and Administrator — with include-relationships linking scanning, sound recording and reporting.



A minimal, trust-building visual language

Brand identity centres on a professional blue (#2563EB) supported by a six-colour palette, the Inter font family, and a consistent grid — kept deliberately clean and easy to implement in Flutter. Sixteen screens were mocked up in total; the core flow is shown here.

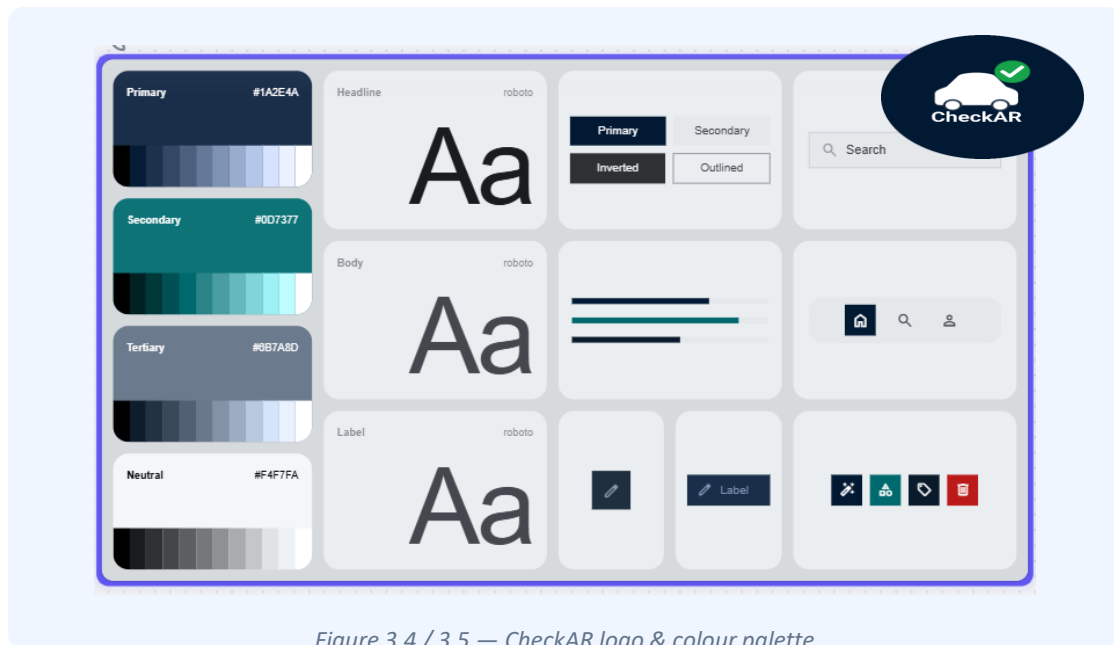
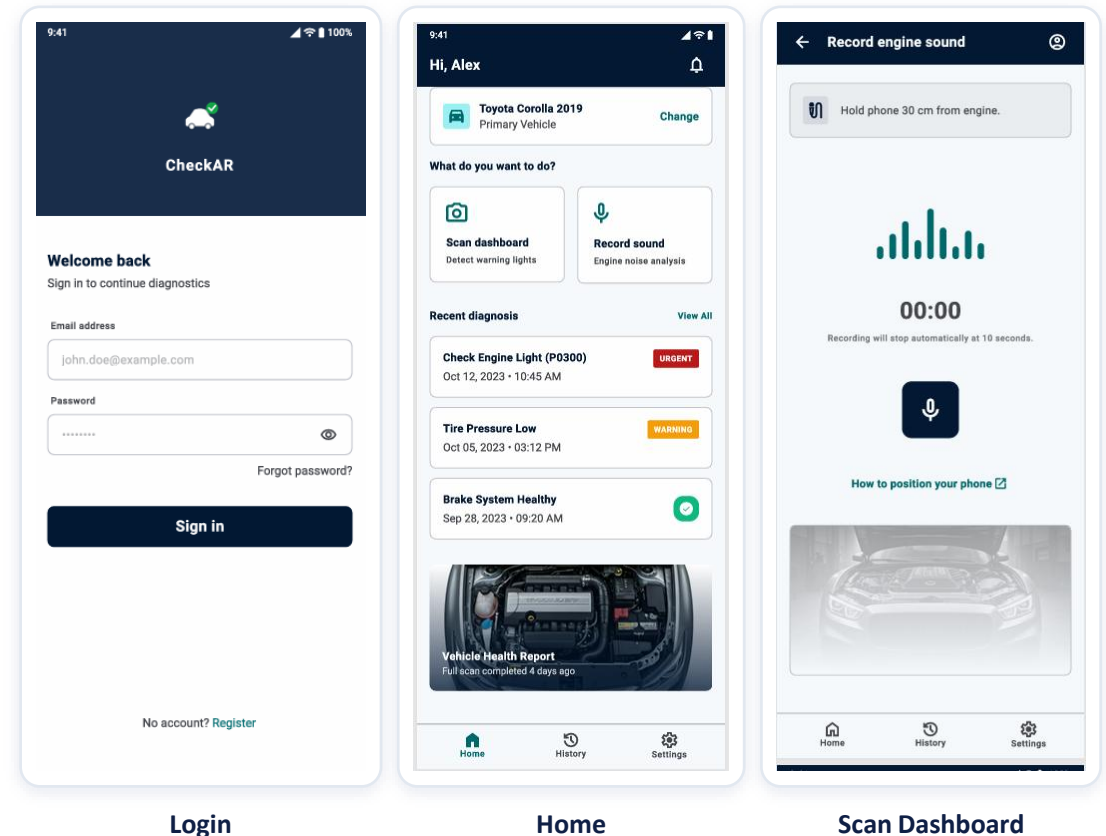


Figure 3.4 / 3.5 — CheckAR logo & colour palette



Twelve-entity data model, three-layer backend

KEY ENTITIES

Users · User_Profiles · Diagnostic_Records · Diagnostic_Results · Warning_Lights · Engine_Sounds
· Maintenance_Records · Mechanics · Services · Tutorials · ML_Models · Audit_Logs

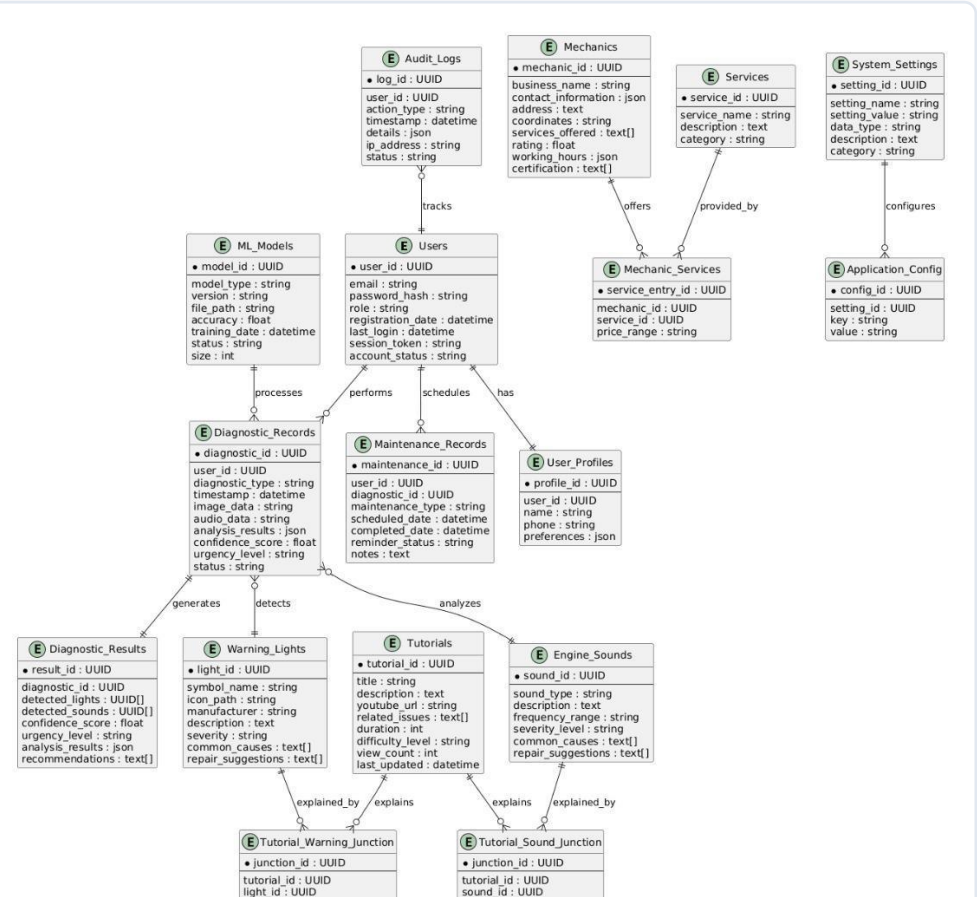


Figure 3.12: CheckAR Entity-Relationship Diagram

Three-Layer Architecture

Flutter mobile client

↓ REST + JWT

Django / DRF backend API

↓ ORM

PostgreSQL relational database

Technology stack and progress to date



Flutter (Dart)

Cross-platform front-end



Django + DRF

Backend & REST API



PostgreSQL

Relational database



TensorFlow

Image & audio ML models



JWT Auth

Secure sessions



Docker

Containerized deployment

STATUS: DESIGN PHASE COMPLETE

Objectives 1–6 (research, requirements, SRS, architecture, UI/UX, database design) are fully satisfied. The general objective — a working diagnostic app — awaits the implementation and testing phase, planned via prototype → alpha → beta → production testing with the original interview/survey participants.

A complete blueprint, ready to build

CheckAR delivers an evidence-based, stakeholder-validated design for hardware-free vehicle diagnostics — grounded in real interviews and surveys with Cameroonian car owners and mechanics.

- 1 Implement the Flutter client and Django/DRF backend
- 2 Train & integrate TensorFlow models for image and sound classification
- 3 Run unit, integration, and user-acceptance testing
- 4 Deploy via Docker and beta-test the Android app with real users